

NS3 Project: TCP with Faster Recovery

CSE-322

Submitted by

Shemanty Mahjabin

ID: 2105091

Contents

1	Overview of the proposed Algorithm	4
2	Implementation in ns-3	5
2.1	Implemented files	5
2.2	Common implementation structure	5
2.3	Where the implementations differ	6
3	SIMULATION RESULTS	7
4	Losses Caused by Bursty Traffic	7
4.1	Network Topology	7
4.2	Goodput vs Number of Connections	8
4.2.1	Parameters under variation	8
4.2.2	Result	8
4.2.3	Graph	8
4.2.4	Summary Findings	9
4.3	Goodput vs Bottleneck Bandwidth	9
4.3.1	Parameters under variation	9
4.3.2	Result	9
4.3.3	Graph	9
4.3.4	Summary Findings	10
4.4	Goodput vs RTT	10
4.4.1	Parameters under variation	10
4.4.2	Result	10
4.4.3	Graph	11
4.4.4	Summary Findings	11
5	Random Losses	11
5.1	Network Topology	11
5.2	Goodput vs Number of Connections	12
5.2.1	Parameters under variation	12
5.2.2	Result	12
5.2.3	Graph	13
5.2.4	Summary Findings	13
5.3	Goodput vs Bottleneck Bandwidth	13
5.3.1	Parameters under variation	13
5.3.2	Result	13
5.3.3	Graph	14
5.3.4	Summary Findings	14
5.4	Goodput vs RTT	14
5.4.1	Parameters under variation	14
5.4.2	Result	15
5.4.3	Graph	15
5.4.4	Summary Findings	15

5.5	Throughput vs time	16
5.5.1	Parameters under variation	16
5.5.2	Result	16
5.5.3	Graph	16
5.5.4	Summary Findings	16
6	802.11 Mobile Network Topology	17
6.1	Network Topology	17
6.2	Varying Number of Nodes	17
6.2.1	Parameters under variation	17
6.2.2	Result	17
6.2.3	Graph	18
6.3	Varying Number of Flows	19
6.3.1	Parameters under variation	19
6.3.2	Result	19
6.3.3	Graph	21
6.4	Varying Number of Packets	21
6.4.1	Parameters under variation	21
6.4.2	Result	22
6.4.3	Graph	23
6.5	Varying Speed of Nodes	23
6.5.1	Parameters under variation	23
6.5.2	Result	24
6.5.3	Graph	25
6.6	Summary Findings	25
7	Wireless 802.15.4 Static Network Topology	26
7.1	Network Topology	26
7.2	Varying Number of Nodes	26
7.2.1	Parameters under Varying	26
7.2.2	Result	27
7.2.3	Graph	29
7.3	Varying Number of Flows	29
7.3.1	Parameters under Varying	29
7.3.2	Result	30
7.3.3	Graph	32
7.4	Varying Number of Packets	32
7.4.1	Parameters under Varying	32
7.4.2	Result	33
7.4.3	Graph	35
7.5	Varying Coverage Area	35
7.5.1	Parameters under Varying	35
7.5.2	Result	36
7.5.3	Graph	38
7.6	Summary Findings	38

1 Overview of the proposed Algorithm

My proposed algorithm is a congestion control algorithm “TCP Faster Recovery”. The algorithm is an improvement upon the TCP Reno algorithm.

The algorithm proposes a new way to calculate the Slow Start Threshold. Upon finding a “Triple Dupack”, TCP Reno algorithm sets the *sssthresh* to *cwind*/2 i.e. half the output rate when a triple dupack was found or the timer expired.

TCP-Faster Recovery algorithm has a cleverer approach, instead of “blindly” reducing the congestion window, it uses a **bandwidth estimate** value to set the new *sssthresh* and compute the congestion window.

Estimation of bandwidth:

TCP-Faster Recovery estimates the bandwidth upon receiving each ACK. This bandwidth estimation “ideally” converges into the actual bandwidth between the source and destination. The pseudocode:

```
if (ACK is received) {
    sample_BWE = pkt_size*8/(now - lastacktime);
    BWE = BWE*alpha + sample_BWE*(1 - alpha);
}
```

Calculation of *sssthresh*:

The estimated bandwidth is later used to calculate the *sssthresh*. Pseudocode:

- triple duplicate ACKS:
 $sssthresh = (BWE * RTT_{min})/a$
 $CWIN = sssthresh$
- coarse timeout expiration:
 $sssthresh = (BWE * RTT_{min})/a$
 $CWIN = 1$

Congestion Avoidance:

In congestion avoidance state, the GFR-TCP or “Gradual Faster Recovery TCP” algorithm improves upon the TCP Reno algorithm. It eliminates the linear increase of the congestion window and introduces a way to recognize when the output can be safely increased. The algorithm uses the estimated bandwidth calculated before to try and increase the *sssthresh* safely so that the network can enter slow start mode and increase the congestion window towards the available bandwidth faster than the linear way.

```
If (CWIN > sssthresh) AND (CWIN < BWE*RTT_min)
then
    sssthresh += (BWE*RTT_min-sssthresh)/2;
```

By using these improvements, the proposed algorithm performs significantly better than the TCP NewReno algorithm.

2 Implementation in ns-3

To implement the proposed algorithms in the ns-3 simulator, I added six files in total. I implemented the new logic as separate congestion control classes derived from `TcpNewReno`.

2.1 Implemented files

The following files were added in `src/internet/model/`:

- `tcp-fr.h`, `tcp-fr.cc`
- `tcp-fr-gfr.h`, `tcp-fr-gfr.cc`
- `tcp-fr-gfr-jitter.h`, `tcp-fr-gfr-jitter.cc`

2.2 Common implementation structure

All three variants share the same main design idea. They inherit from `TcpNewReno` and preserve the normal NewReno behavior as the base, while replacing the parts related to bandwidth-aware recovery. In all three implementations, the following common state is maintained:

- a smoothing factor `m_alpha` for bandwidth estimation,
- a reduction factor `m_a`,
- a bandwidth estimate variable,
- the time of the previous ACK,
- the minimum RTT observed so far,
- and a flag for handling the first ACK sample.

The same core member functions are also present in all variants:

- `PktsAacked()` for updating the bandwidth estimate. Bandwidth estimation according to the paper implemented here,
- `CongestionStateSet()` for reacting to loss or recovery states,
- `IncreaseWindow()` for slow start and congestion avoidance behavior,
- `GetSsThresh()` for computing the slow start threshold,
- `Fork()` for copying the congestion control object.

2.3 Where the implementations differ

Although the three variants share the same recovery base, they differ in the congestion avoidance part.

FR-TCP

The FR implementation is the simplest of the three. It includes the bandwidth estimation and the bandwidth-based `ssthresh` calculation, but after recovery it still uses the normal NewReno slow start and congestion avoidance behavior. In other words, FR-TCP changes how the sender reacts to losses, but it does not introduce any extra periodic mechanism for increasing `ssthresh` later.

GFR-TCP

The GFR implementation keeps everything from FR, but adds one extra idea from the paper: a periodic congestion avoidance event. Once the sender is already in congestion avoidance, and if the current congestion window is larger than `ssthresh` but still smaller than the estimated bandwidth-delay product, a timer is started. In my implementation, this timer is scheduled every 500 ms.

When the timer fires, the sender checks the GFR condition and may increase `ssthresh` using

Listing 1: Periodic GFR threshold increase

```
if ((tcb->m_cWnd > tcb->m_ssThresh) && (tcb->m_cWnd < bweRttProduct))
{
    tcb->m_ssThresh += (bweRttProduct - tcb->m_ssThresh) / 2;
}
```

This makes GFR more aggressive than FR, because it tries to move the connection back toward the available bandwidth faster than plain linear congestion avoidance.

Jitter-aware GFR

The jitter-aware version is based on GFR, but adds a protection mechanism. Besides bandwidth estimation and minimum RTT tracking, it also measures how much the current RTT differs from the minimum RTT. This difference is treated as a queueing-delay or jitter signal, and it is smoothed over time using another EWMA.

Listing 2: Jitter-aware safety check

```
bool
TcpFrGfrJitter::AllowGfrBoost() const
{
    if (!m_hasJitterSample)
    {
        return true;
    }
}
```

```

    }
    return Seconds(m_smoothedJitterSeconds) <= m_jitterThreshold;
}

```

The key difference is that the GFR boost is only allowed when the smoothed jitter remains below a threshold. If the RTT variation becomes too high, the implementation suppresses the periodic GFR boost. This means the jitter-aware version keeps the FR/GFR recovery logic, but avoids becoming too aggressive when the path looks unstable.

3 SIMULATION RESULTS

4 Losses Caused by Bursty Traffic

4.1 Network Topology

I reproduced the bursty UDP traffic experiment in **ns-3** using a **dumbbell topology** as mentioned in the paper. In this topology, multiple TCP sender nodes are connected to router *R1*, while multiple TCP receiver nodes are connected to router *R2*. The two routers are connected by a single bottleneck link, which is where congestion mainly occurs. In addition to the TCP flows, one UDP source and one UDP sink were added so that bursty UDP traffic also crosses the same bottleneck and competes with TCP traffic.

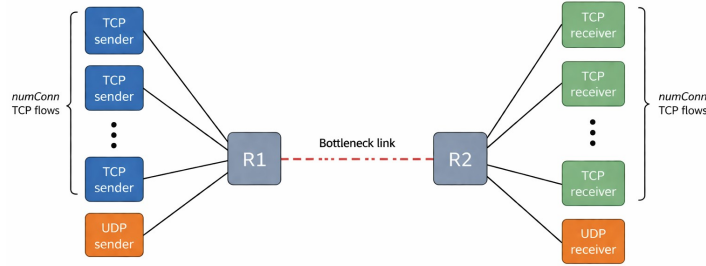


Figure 1: Dumbbell topology used in the ns-3 simulation.

The access links were configured with high bandwidth so that they do not become the limiting factor, while the bottleneck link was varied depending on the experiment. The UDP traffic was generated using an On/Off application with **20 s On** and **20 s Off**. During the On period, the UDP flow consumes

about **90%** of the bottleneck bandwidth, which creates bursty losses and makes the TCP competition more challenging. Each simulation was run for **200 seconds**. The TCP variants compared in this work were **FR**, **GFR**, **GA-GFR**, **Reno**, and **Sack**, and the main performance metric was the average **goodput**.

4.2 Goodput vs Number of Connections

4.2.1 Parameters under variation

I varied the number of TCP connections while keeping the bottleneck bandwidth fixed at 45 Mb/s and the RTT fixed at 0.5 s.

4.2.2 Result

Connections	Reno	Sack	FR	GFR	JA-GFR
1	3.955120	5.331350	16.507000	21.107800	23.734300
5	1.124700	5.642910	5.446230	4.913980	5.060760
10	1.733230	2.985520	2.497780	2.621340	2.885270
20	1.167780	1.562280	1.677000	1.637370	1.644460
30	0.791964	1.095360	1.136430	1.127810	1.158130

Table 1: Goodput vs Number of Connections

4.2.3 Graph

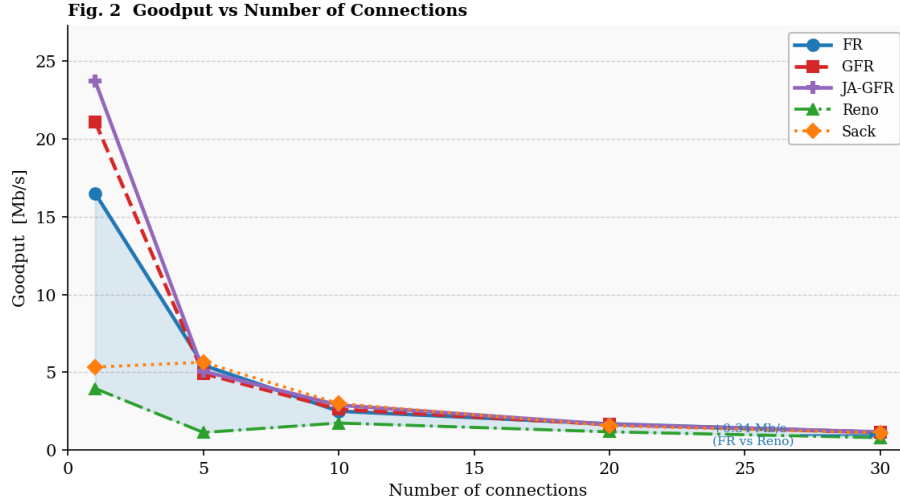


Figure 2: Goodput as a function of number of concurrent connections

4.2.4 Summary Findings

In the first experiment, where the number of connections was increased, **JA-GFR** performed the best when there was only one TCP connection, and **GFR,FR** also showed very strong performance. As the number of connections increased, the goodput of all algorithms dropped significantly, which is expected because more flows had to share the same bottleneck while bursty UDP traffic was also present.

4.3 Goodput vs Bottleneck Bandwidth

4.3.1 Parameters under variation

I varied the bottleneck bandwidth while using 10 TCP connections and an RTT of 0.5 s.

4.3.2 Result

Bandwidth	Reno	Sack	FR	GFR	JA-GFR
1	0.066641	0.082365	0.083154	0.083592	0.083154
45	1.733230	2.985520	2.497780	2.694570	2.885270
150	2.504200	9.042730	6.120930	8.270500	9.085720

Table 2: Goodput vs Bandwidth

4.3.3 Graph

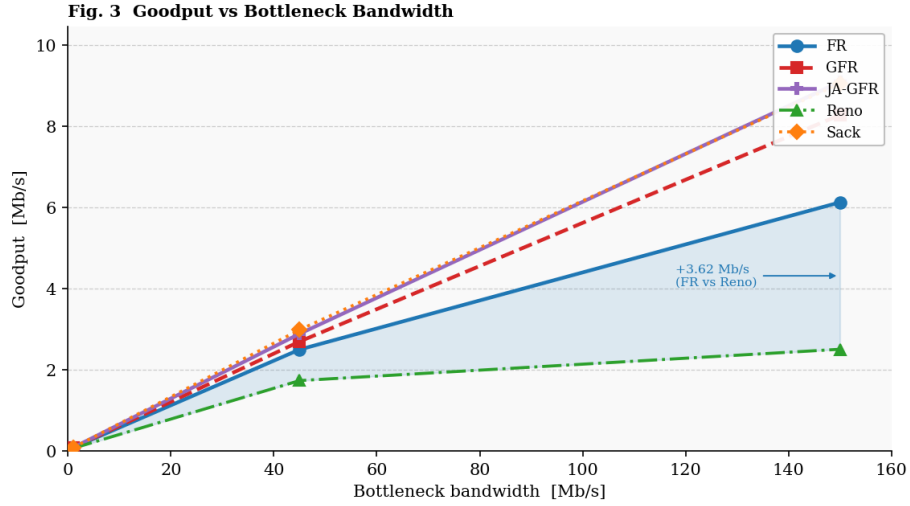


Figure 3: Goodput vs Bottleneck Bandwidth

4.3.4 Summary Findings

In the bandwidth experiment, all variants showed nearly the same very low goodput at the smallest bandwidth. At 45 Mb/s, the results became more separated, with **Sack** and **JA-GFR** achieving the highest goodput, followed by **GFR**, then **FR**, while **Reno** remained the lowest. At the highest tested bandwidth of 150 Mb/s, **JA-GFR** achieved the best goodput, very slightly ahead of **Sack**, with **GFR** next, and **FR** still performing better than Reno. **JA-GFR** performs best because, it applies a jitter-aware GFR mechanism that uses bandwidth-based recovery when the path is stable but suppresses overly aggressive window growth when RTT jitter increases. This makes JA-GFR more effective at exploiting high bandwidth while avoiding extra congestion under bursty interference.

4.4 Goodput vs RTT

4.4.1 Parameters under variation

I varied the RTT while keeping 10 TCP connections and a bottleneck bandwidth of 45 Mb/s.

4.4.2 Result

RTT	Reno	Sack	FR	GFR	JA-GFR
0.01	4.107800	4.104990	3.501180	3.501180	3.501180
0.05	3.852560	3.883350	3.858240	3.865370	3.858240
0.10	3.454540	3.829960	3.647710	3.763480	3.647710
0.50	1.733230	2.985520	2.497780	2.621340	2.885270
1.00	0.680369	2.088530	2.405880	2.186710	2.650970

Table 3: Goodput vs RTT

4.4.3 Graph

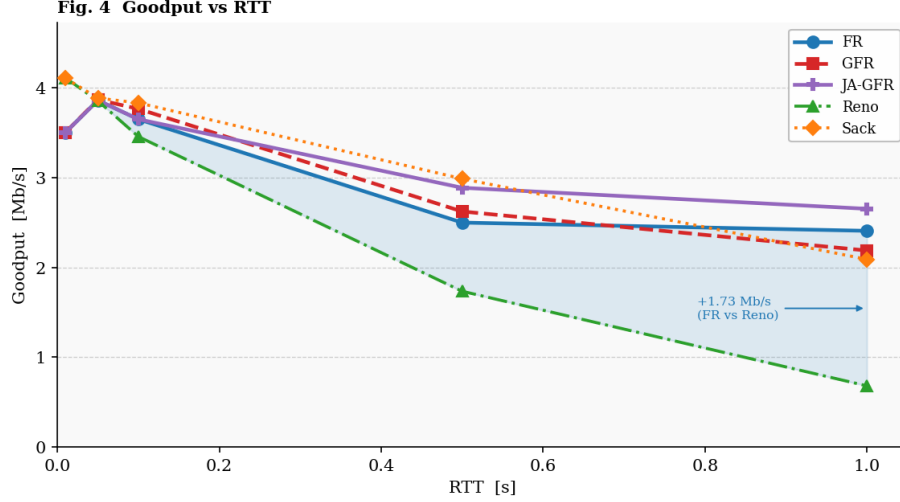


Figure 4: Goodput vs RTT

4.4.4 Summary Findings

In the RTT experiment, for very small RTT values, Reno and Sack achieved slightly better goodput, which may indicate that the more aggressive recovery behavior of FR and GFR is not always helpful in short-delay networks. However, as RTT increased, **FR became more effective**. At 1.0 s RTT, FR achieved the highest goodput among all the tested variants. This is an important result because it supports the general idea that Faster Recovery methods are more suitable for long-delay links, where recovering quickly from losses is more important. GFR did not always dominate in this experiment, but both Faster Recovery variants still showed that they can be beneficial when the RTT becomes large.

5 Random Losses

5.1 Network Topology

I implemented the TCP lossy-link experiment in `ns-3` using a **dumbbell topology**. Multiple TCP sender nodes are connected to a left gateway router, and their traffic traverses a shared *lossy bottleneck link* toward a right gateway and a receiver side. The access links are configured with high bandwidth so that the dominant limitation comes from the bottleneck link and the losses injected on it.

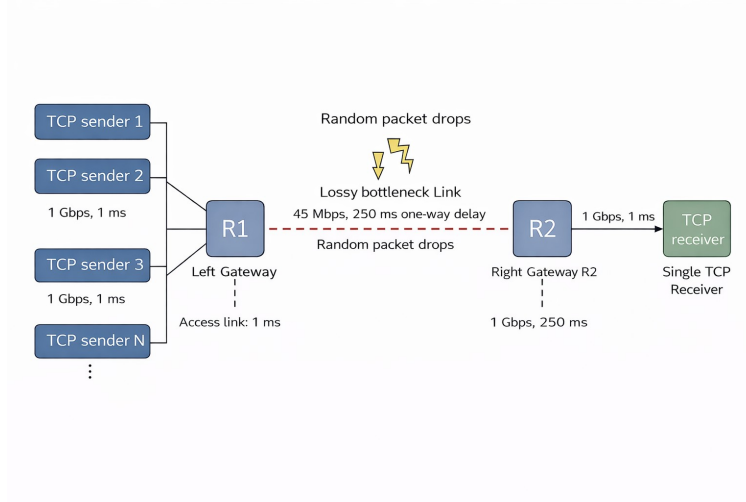


Figure 5: Dumbbell topology used for the lossy-link simulation.

In this setup, packet loss is applied only on the bottleneck path to emulate a lossy network. By default, a burst-error process is used; alternatively, a rate-based packet error model can be enabled. Each simulation is run for **200 seconds**. The evaluated TCP congestion-control variants are **FR**, **GFR**, **Reno**, and **JitterAdaptiveGFR**. **SACK** is treated as a TCP option (enabled/disabled), not as a separate congestion-control variant. The primary performance metric is **average goodput**.

5.2 Goodput vs Number of Connections

5.2.1 Parameters under variation

I varied the number of TCP connections while keeping the bottleneck bandwidth fixed at 45 Mb/s and the RTT fixed at 0.5 s.

5.2.2 Result

Connections	Reno	Sack	FR	GFR	JA-GFR
1	1.127170	1.436320	2.011370	2.011370	2.011370
5	0.708331	0.784838	2.008750	2.008750	2.008750
10	0.930339	1.011410	2.055790	2.055790	2.055790
20	0.987742	0.934712	1.900620	1.900620	1.900620
30	0.837644	0.838932	1.096160	1.141260	1.110570

Table 4: Goodput vs Number of Connections

5.2.3 Graph

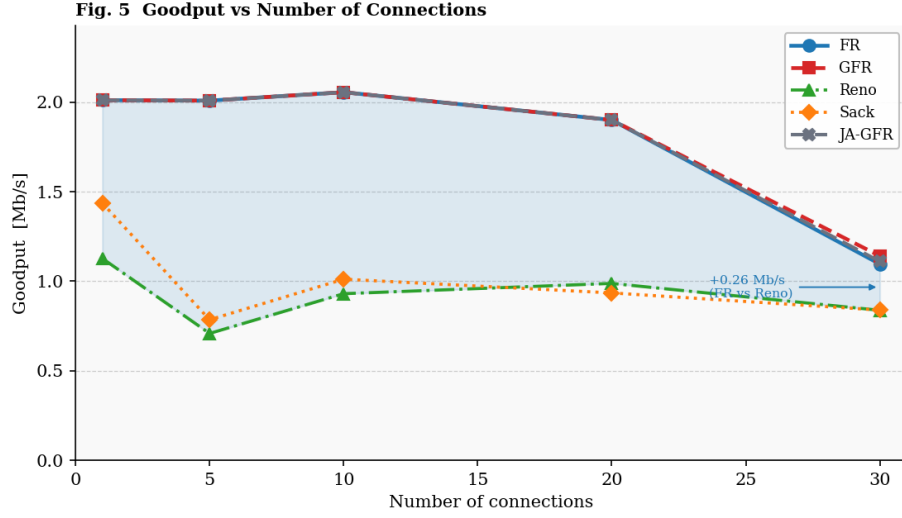


Figure 6: Goodput vs connections.

5.2.4 Summary Findings

From my results, **FR**, **GFR** and **JA-GFR** are clearly the strongest algorithms in the lossy-link scenario, even when the number of connections becomes large, they still remain well above Reno and Sack. This suggests that the Faster Recovery based variants handle repeated losses better when many flows compete at the same bottleneck.

5.3 Goodput vs Bottleneck Bandwidth

5.3.1 Parameters under variation

I varied the bottleneck bandwidth while using 10 TCP connections and an RTT of 0.5 s.

5.3.2 Result

Bandwidth	Reno	Sack	FR	GFR	JA-GFR
2	0.195179	0.197080	0.187380	0.184975	0.187380
40	1.022460	0.985916	2.077270	2.077270	2.077270
150	0.958696	0.972756	2.173950	2.173950	2.173950

Table 5: Goodput vs Bandwidth

5.3.3 Graph

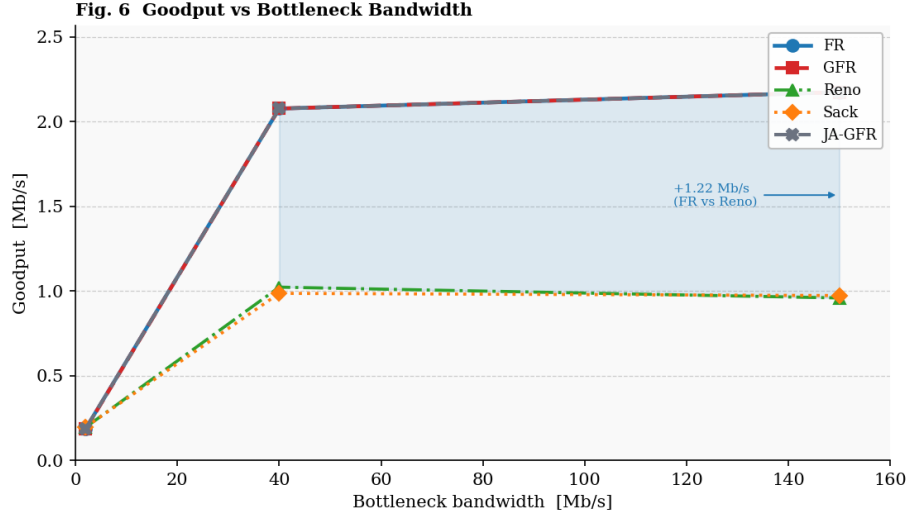


Figure 7: Goodput versus bottleneck bandwidth.

5.3.4 Summary Findings

A similar trend appears in Figure 7. As the bottleneck bandwidth increases, **FR** and **GFR** and **JA-GFR** scale much better than Reno and Sack. Reno and Sack stay around roughly 1 Mb/s, while FR and GFR rise much higher and then remain almost flat at the top. This is a strong result in favor of Faster Recovery, because it shows that the aggressive recovery mechanism is making better use of the available bandwidth in a lossy environment.

5.4 Goodput vs RTT

5.4.1 Parameters under variation

I varied the RTT while keeping 10 TCP connections and a bottleneck bandwidth of 45 Mb/s.

5.4.2 Result

RTT	Reno	Sack	FR	GFR	JA-GFR
0.01	4.486700	4.486460	4.486760	4.486780	4.486760
0.05	4.446780	4.435590	4.476750	4.476010	4.476750
0.10	4.009390	3.867770	4.349540	4.201540	4.348790
0.50	0.930339	1.011410	2.055790	2.055790	2.055790
1.00	0.438629	0.440792	0.870778	0.870778	0.870778

Table 6: Goodput vs RTT

5.4.3 Graph

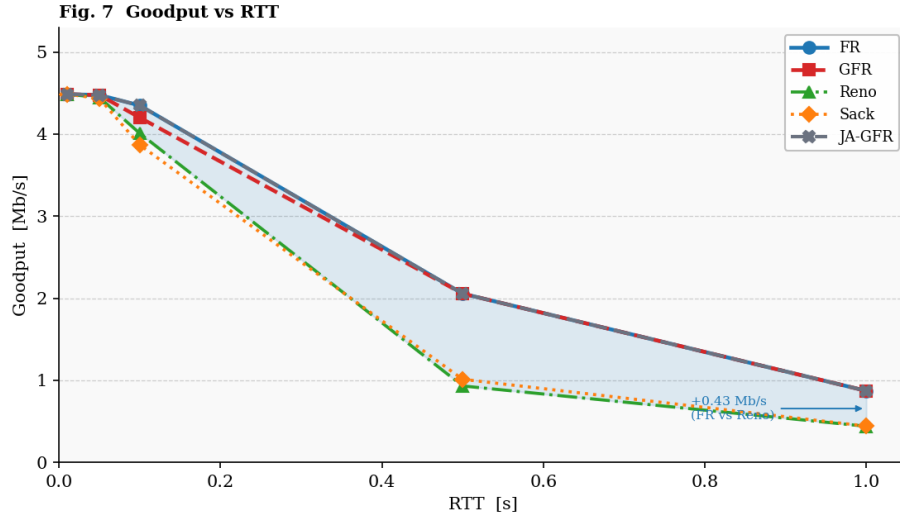


Figure 8: Goodput versus RTT.

5.4.4 Summary Findings

In Figure 8, FR, GFR, JA-GFR again perform best across almost the whole RTT range. The gap becomes even more obvious at larger RTT values, where Reno and Sack fall more quickly. This means the Faster Recovery variants are especially helpful when the network has both delay and random losses, which is also the kind of case where faster recovery should matter the most.

5.5 Throughput vs time

5.5.1 Parameters under variation

A 5Mb/s link with a 00ms propagation delay. The throughputs of a single *of0* connection for each version are plotted as a function of time.

5.5.2 Result

5.5.3 Graph

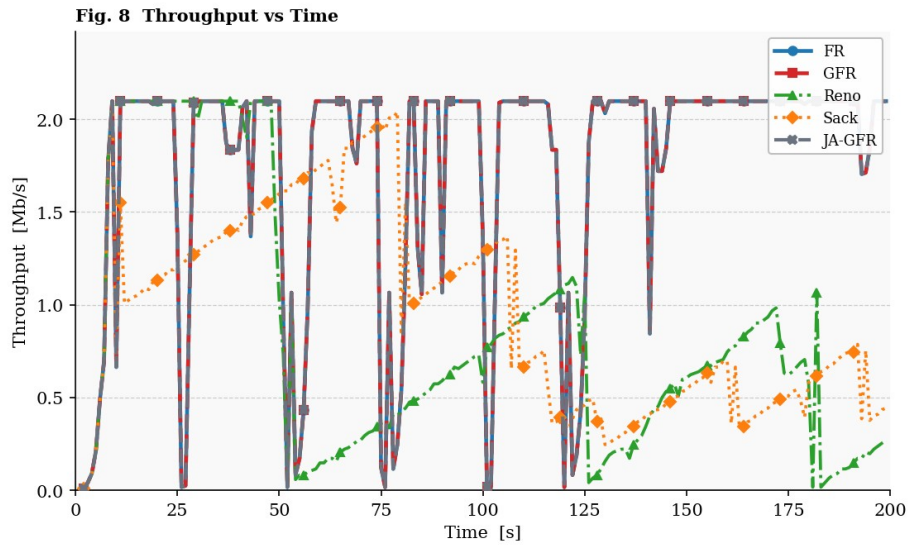


Figure 9: Throughput versus time for the lossy-link case.

5.5.4 Summary Findings

Finally, Figure 9 shows the throughput of one flow over time. Here, **FR** and **GFR** recover very quickly after loss events and spend most of the simulation near the top throughput level. Reno, on the other hand, drops much more and takes longer to recover. So even though my results are not numerically identical to the paper, the general conclusion from my ns-3 experiments is that **FR and GFR behave much better than Reno and Sack in the lossy-link case**, especially when the bandwidth or RTT becomes larger.

6 802.11 Mobile Network Topology

6.1 Network Topology

As per the project checklist, I tested them in a wireless mobile network. The simulation was built as an **IEEE 802.11 ad hoc MANET** in **ns-3**, where all nodes communicate over Wi-Fi and routing is handled by AODV. In the mobile case, the nodes follow a Random Waypoint mobility model, and multiple TCP flows are created between randomly selected source-destination pairs. The main metrics collected were throughput, mean delay, packet delivery ratio (PDR), drop ratio, and total energy consumption.

6.2 Varying Number of Nodes

6.2.1 Parameters under variation

The number of nodes was varied as 20, 40, 60, and 80, while the number of packets per second was fixed at 300, the number of flows was fixed at 30, and the speed of nodes was fixed at 20 m/s.

6.2.2 Result

Nodes	Reno	FR	GFR	JA-GFR
20	10.288653	9.799849	9.642735	9.761193
40	8.747746	8.563023	8.333459	8.563023
60	7.282790	7.340220	7.340220	7.340220
80	7.888143	8.380164	7.731872	8.380164

Table 7: Throughput (Mbps) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	39.432999	64.686849	52.647628	82.859510
40	28.082486	42.873326	36.741033	42.873326
60	16.253928	20.347121	20.347121	20.347121
80	34.912334	37.680436	34.172884	37.680436

Table 8: Mean Delay (ms) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	97.799344	94.324192	94.705033	94.123607
40	97.240666	93.981420	94.929394	93.981420
60	97.733038	94.729505	94.729505	94.729505
80	95.163801	93.754617	93.296170	93.754617

Table 9: Packet Delivery Ratio (%) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	1.270158	3.915845	3.959433	3.682154
40	2.108705	3.650020	3.576435	3.650020
60	1.268371	3.616931	3.616931	3.616931
80	3.471493	5.063461	5.411569	5.063461

Table 10: Packet Drop Ratio (%) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	343.406651	343.440985	343.220552	343.718822
40	679.599083	679.800577	678.743176	679.800577
60	1014.862524	1014.864800	1014.864800	1014.864800
80	1352.869549	1354.518884	1353.460461	1354.518884

Table 11: Total Energy Consumption (J) vs Number of Nodes

6.2.3 Graph

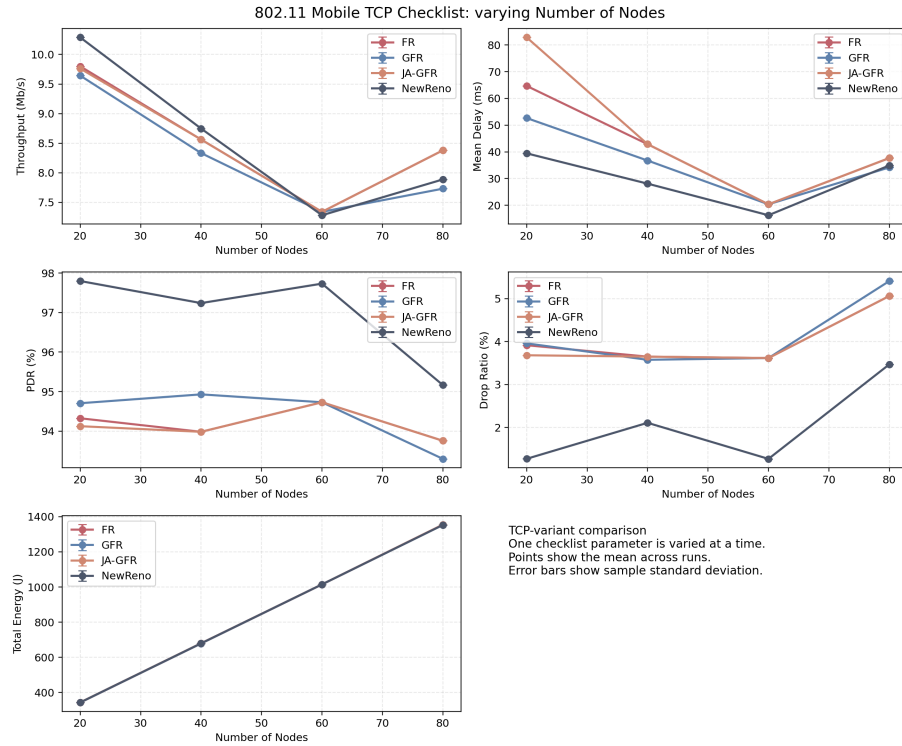


Figure 10: Performance comparison while varying the number of nodes.

6.3 Varying Number of Flows

6.3.1 Parameters under variation

The number of flows was varied as 10, 20, 30, 40, and 50, while the number of nodes was fixed at 40, the number of packets per second was fixed at 300, and the speed of nodes was fixed at 20 m/s.

6.3.2 Result

Flows	Reno	FR	GFR	JA-GFR
10	3.940593	3.792574	3.792574	3.792574
20	6.965269	7.344446	7.144474	7.344446
30	8.747746	8.563023	8.333459	8.563023
40	9.221252	8.372890	8.372890	8.372890
50	9.842438	8.601370	8.567059	8.399019

Table 12: Throughput (Mbps) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	41.475249	42.645573	42.645573	42.645573
20	23.861841	31.524786	35.549184	31.524786
30	28.082486	42.873326	36.741033	42.873326
40	28.560907	31.772290	31.772290	31.772290
50	27.380762	35.915959	38.039479	50.504536

Table 13: Mean Delay (ms) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	95.997093	96.291780	96.291780	96.291780
20	95.874101	95.989769	96.086406	95.989769
30	97.240666	93.981420	94.929394	93.981420
40	97.215307	89.861337	89.861337	89.861337
50	97.894863	89.250585	89.160595	87.797713

Table 14: Packet Delivery Ratio (%) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	2.712610	2.468189	2.468189	2.468189
20	1.795528	3.190366	3.251298	3.190366
30	2.108705	3.650020	3.576435	3.650020
40	1.805049	9.143996	9.143996	9.143996
50	1.080793	10.080608	10.378297	11.361167

Table 15: Packet Drop Ratio (%) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	670.792504	668.941270	668.941270	668.941270
20	676.756548	676.634250	676.457657	676.634250
30	679.599083	679.800577	678.743176	679.800577
40	680.010846	681.843793	681.843793	681.843793
50	681.030821	683.304626	683.969605	685.702713

Table 16: Total Energy Consumption (J) vs Number of Flows

6.3.3 Graph

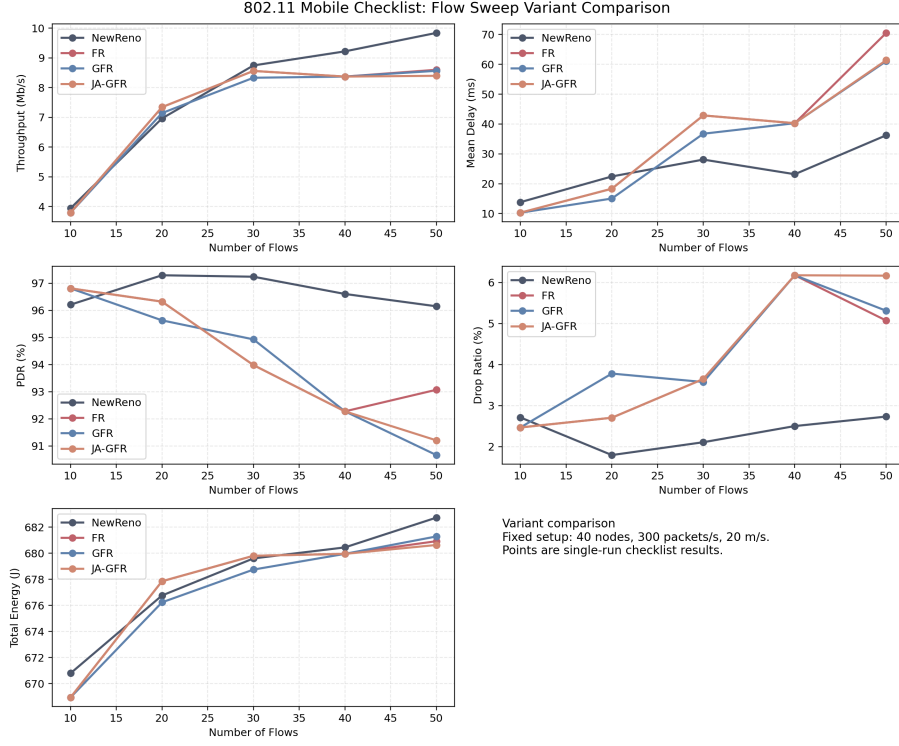


Figure 11: Performance comparison while varying the number of flows.

6.4 Varying Number of Packets

6.4.1 Parameters under variation

The number of packets per second was varied as 100, 200, 300, 400, and 500, while the number of nodes was fixed at 40, the number of flows was fixed at 30, and the speed of nodes was fixed at 20 m/s.

6.4.2 Result

Packets/s	Reno	FR	GFR	JA-GFR
100	5.480452	5.210342	5.174458	5.174458
200	7.507731	6.573532	6.787290	6.573532
300	8.747746	8.563023	8.333459	8.563023
400	9.332416	8.480770	8.293099	8.470569
500	9.791430	9.219202	9.368484	9.219202

Table 17: Throughput (Mbps) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	12.953529	21.913862	27.377796	27.377796
200	18.155813	33.465263	40.495524	33.465263
300	28.082486	42.873326	36.741033	42.873326
400	24.822897	42.460054	39.293212	50.455215
500	20.251643	58.615129	56.479747	58.615129

Table 18: Mean Delay (ms) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	97.055234	95.634484	92.358697	92.358697
200	97.482855	92.021835	92.742696	92.021835
300	97.240666	93.981420	94.929394	93.981420
400	97.494041	92.900362	93.036099	93.588936
500	97.270775	91.799211	93.210264	91.799211

Table 19: Packet Delivery Ratio (%) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	2.175413	3.403343	5.165499	5.165499
200	1.862531	6.407726	5.978773	6.407726
300	2.108705	3.650020	3.576435	3.650020
400	1.630912	6.455113	6.000808	5.787536
500	2.099863	5.749223	4.857066	5.749223

Table 20: Packet Drop Ratio (%) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	674.755515	673.923891	674.138648	674.138648
200	676.601667	676.736677	676.937555	676.736677
300	679.599083	679.800577	678.743176	679.800577
400	679.409368	679.502358	678.686931	679.452977
500	678.864054	680.108797	680.033425	680.108797

Table 21: Total Energy Consumption (J) vs Packets per Second

6.4.3 Graph

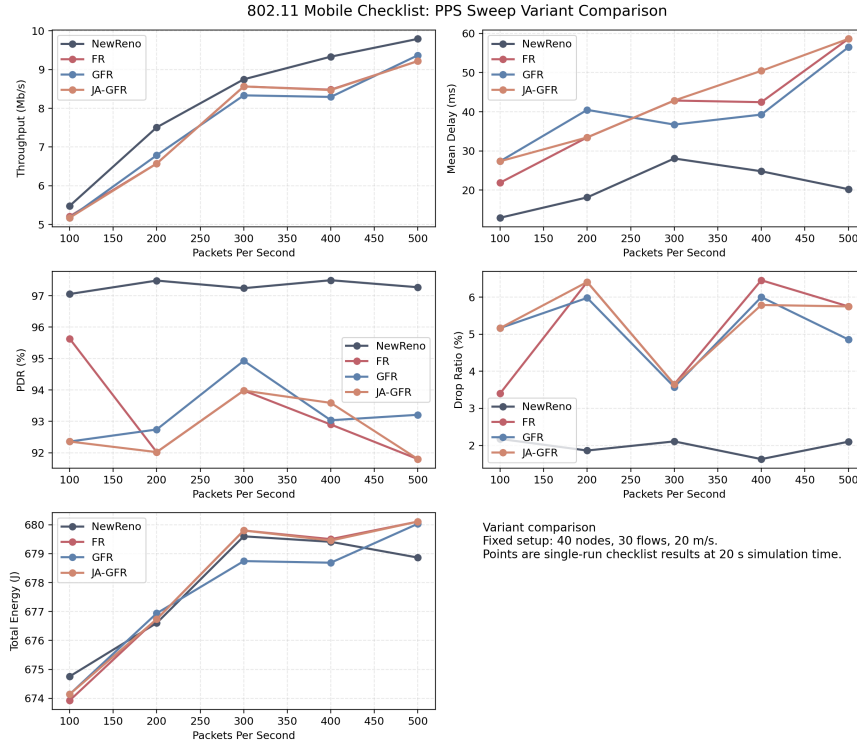


Figure 12: Performance comparison while varying packets per second.

6.5 Varying Speed of Nodes

6.5.1 Parameters under variation

The speed of nodes was varied as 5, 10, 15, 20, and 25 m/s, while the number of nodes was fixed at 40, the number of flows was fixed at 30, and the number of packets per second was fixed at 300.

6.5.2 Result

Speed (m/s)	Reno	FR	GFR	JA-GFR
5	9.893442	9.413312	9.251422	9.655373
10	9.933843	8.991078	8.991078	8.991078
15	8.973781	8.875849	8.726716	8.875849
20	8.747746	8.563023	8.333459	8.563023
25	6.558357	7.368979	7.766912	7.368979

Table 22: Throughput (Mbps) vs Node Speed

Speed (m/s)	Reno	FR	GFR	JA-GFR
5	24.513730	28.510972	36.008084	27.565150
10	17.398360	25.719428	25.719428	25.719428
15	18.412248	30.082621	35.755498	30.082621
20	28.082486	42.873326	36.741033	42.873326
25	25.285791	21.913326	35.143219	21.913326

Table 23: Mean Delay (ms) vs Node Speed

Speed (m/s)	Reno	FR	GFR	JA-GFR
5	97.575488	93.794485	93.247618	93.218374
10	97.769176	93.252412	93.252412	93.252412
15	98.184087	93.771604	93.141338	93.771604
20	97.240666	93.981420	94.929394	93.981420
25	96.503620	95.462851	94.627903	95.462851

Table 24: Packet Delivery Ratio (%) vs Node Speed

Speed (m/s)	Reno	FR	GFR	JA-GFR
5	1.897573	5.601412	5.829678	5.941168
10	1.332585	5.515095	5.515095	5.515095
15	1.247825	5.346780	5.468350	5.346780
20	2.108705	3.650020	3.576435	3.650020
25	2.326505	3.475759	3.953191	3.475759

Table 25: Packet Drop Ratio (%) vs Node Speed

Speed (m/s)	Reno	FR	GFR	JA-GFR
5	678.764362	679.060988	677.416141	679.437686
10	679.947563	678.869015	678.869015	678.869015
15	679.113818	679.175566	679.172823	679.175566
20	679.599083	679.800577	678.743176	679.800577
25	675.216433	676.531680	677.067874	676.531680

Table 26: Total Energy Consumption (J) vs Node Speed

6.5.3 Graph

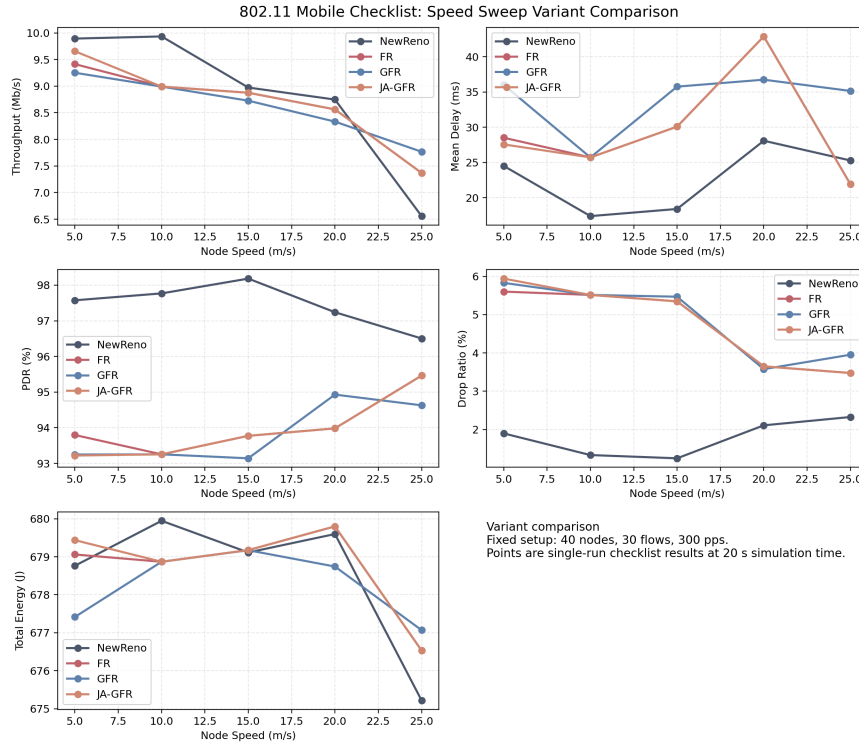


Figure 13: Performance comparison while varying node speed.

6.6 Summary Findings

The wireless mobile results are different from the wired results in the paper, and in many cases TCP NewReno performs better than FR, GFR, and JA-GFR.

- **FR/GFR were designed for wired long-delay links.** They work best when the bandwidth-delay product is large and ACK timing is relatively stable.

- **This topology is a wireless mobile network.** It uses shared Wi-Fi, so performance is affected by contention, backoff, collisions, and interference. Node mobility also causes route and RTT changes.
- **Bandwidth estimation is less reliable in MANETs.** FR and GFR depend on ACK-based bandwidth estimation, but in wireless networks ACK timing becomes noisy because of MAC behavior, retransmissions, and mobility. This can cause *ssthresh* to be set too aggressively or at the wrong time.
- **Wireless losses are different from wired losses.** In the paper, losses are mainly random losses on wired-like paths. But in here, losses and delay variation also come from contention, route changes, and wireless effects, so recovery decisions become less accurate.
- **NewReno can perform better because it is simpler and more conservative.** In a dynamic wireless topology, this conservative behavior can be more stable and may lead to better throughput or PDR.
- **JA-GFR tries to handle jitter, but it still does not consistently beat NewReno.** Wireless variation may be too frequent and too large, so the algorithm either suppresses boosting too often or cannot adapt fast enough.

7 Wireless 802.15.4 Static Network Topology

7.1 Network Topology

To further evaluate the proposed TCP variants, I also tested them in a **static IEEE 802.15.4 wireless network** instead of the wired topology used in the original paper. The simulation was implemented in **ns-3** using **LR-WPAN** devices with **6LoWPAN** enabled. In this setup, the nodes do not move, so the topology remains static throughout the simulation, but communication still takes place over a low-rate wireless multi-hop network. The main performance metrics collected were throughput, mean delay, packet delivery ratio (PDR), drop ratio, and total energy consumption.

7.2 Varying Number of Nodes

7.2.1 Parameters under Varying

Number of nodes was varied as 20, 40, 60, 80, 100 when the number of flows was 50, the packet rate was 400 packets/s, the coverage area was $5 \times Tx_range$, and the simulation time was 20 s.

7.2.2 Result

Nodes	Reno	FR	GFR	JA-GFR
20	0.030	0.028	0.028	0.028
40	0.018	0.018	0.018	0.018
60	0.011	0.011	0.011	0.011
80	0.013	0.014	0.014	0.014
100	0.013	0.012	0.012	0.012

Table 27: Throughput (Mbps) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	738.702	792.654	792.654	792.654
40	1063.923	1050.404	1050.404	1050.404
60	1413.727	1413.727	1413.727	1413.727
80	1018.402	1187.788	1187.788	1187.788
100	1162.647	1167.261	1167.261	1167.261

Table 28: Mean Delay (ms) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	64.382	62.443	62.443	62.443
40	44.482	43.182	43.182	43.182
60	34.433	34.433	34.433	34.433
80	36.525	33.488	33.488	33.488
100	36.398	36.133	36.133	36.133

Table 29: Packet Delivery Ratio (%) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	0.002	0.001	0.001	0.001
40	0.000	0.000	0.000	0.000
60	0.000	0.000	0.000	0.000
80	0.000	0.000	0.000	0.000
100	0.000	0.000	0.000	0.000

Table 30: Mean Throughput per Node (Mbps) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	18.778	19.847	19.847	19.847
40	32.088	34.253	34.253	34.253
60	29.072	29.072	29.072	29.072
80	37.766	37.346	37.346	37.346
100	35.647	37.109	37.109	37.109

Table 31: Packet Drop Ratio (%) vs Number of Nodes

Nodes	Reno	FR	GFR	JA-GFR
20	18.623	18.581	18.581	18.581
40	40.410	40.451	40.451	40.451
60	61.968	61.968	61.968	61.968
80	83.082	82.772	82.772	82.772
100	104.700	104.959	104.959	104.959

Table 32: Total Energy Consumption (J) vs Number of Nodes

7.2.3 Graph

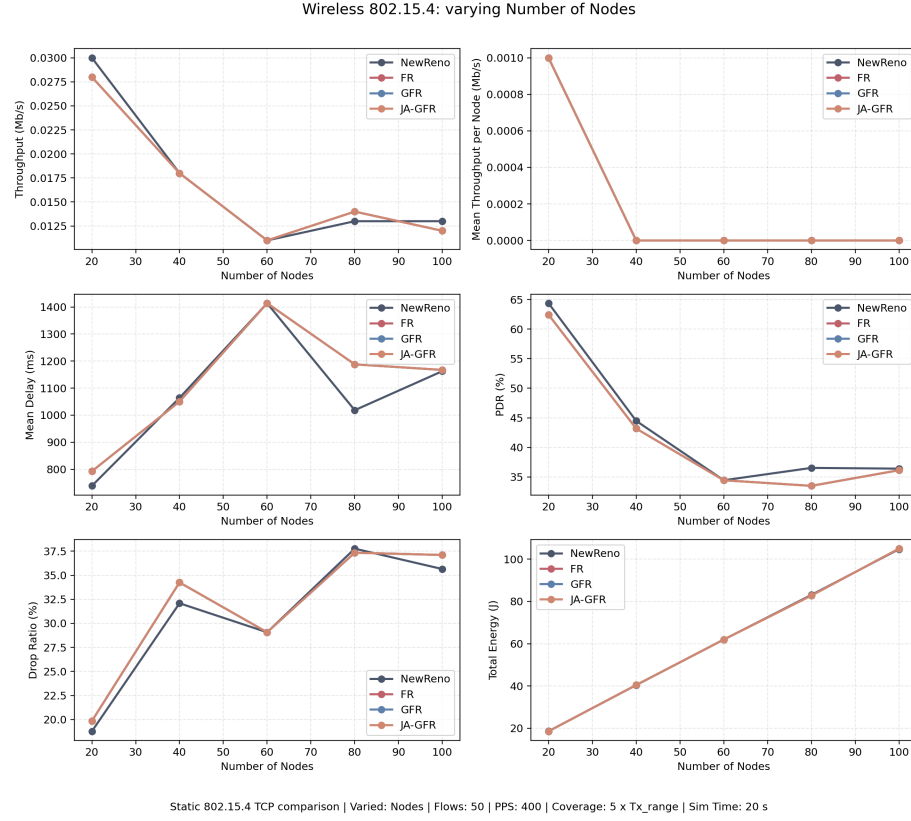


Figure 14: Performance comparison while varying the number of nodes.

7.3 Varying Number of Flows

7.3.1 Parameters under Varying

Number of flows was varied as 10, 20, 30, 40, 50 when the number of nodes was 10, the packet rate was 400 packets/s, the coverage area was $5 \times Tx_range$, and the simulation time was 20 s.

7.3.2 Result

Flows	Reno	FR	GFR	JA-GFR
10	0.003	0.004	0.004	0.004
20	0.019	0.021	0.021	0.021
30	0.033	0.039	0.039	0.039
40	0.057	0.067	0.067	0.067
50	0.073	0.092	0.092	0.092

Table 33: Throughput (Mbps) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	1584.714	1306.170	1306.170	1306.170
20	509.011	641.001	641.001	641.001
30	465.835	1125.521	1125.521	1125.521
40	382.102	1205.054	1205.054	1205.054
50	357.595	363.640	363.640	363.640

Table 34: Mean Delay (ms) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	53.488	55.446	55.446	55.446
20	68.421	65.401	65.401	65.401
30	70.306	65.385	65.385	65.385
40	79.247	62.720	62.720	62.720
50	79.341	80.970	80.970	80.970

Table 35: Packet Delivery Ratio (%) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	0.000	0.000	0.000	0.000
20	0.002	0.002	0.002	0.002
30	0.003	0.004	0.004	0.004
40	0.006	0.007	0.007	0.007
50	0.007	0.009	0.009	0.009

Table 36: Mean Throughput per Node (Mbps) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	26.744	23.762	23.762	23.762
20	15.288	23.840	23.840	23.840
30	13.392	20.979	20.979	20.979
40	10.618	7.807	7.807	7.807
50	10.928	7.939	7.939	7.939

Table 37: Packet Drop Ratio (%) vs Number of Flows

Flows	Reno	FR	GFR	JA-GFR
10	9.828	9.892	9.892	9.892
20	9.336	9.336	9.336	9.336
30	8.820	8.794	8.794	8.794
40	8.271	8.081	8.081	8.081
50	7.720	7.529	7.529	7.529

Table 38: Total Energy Consumption (J) vs Number of Flows

7.3.3 Graph

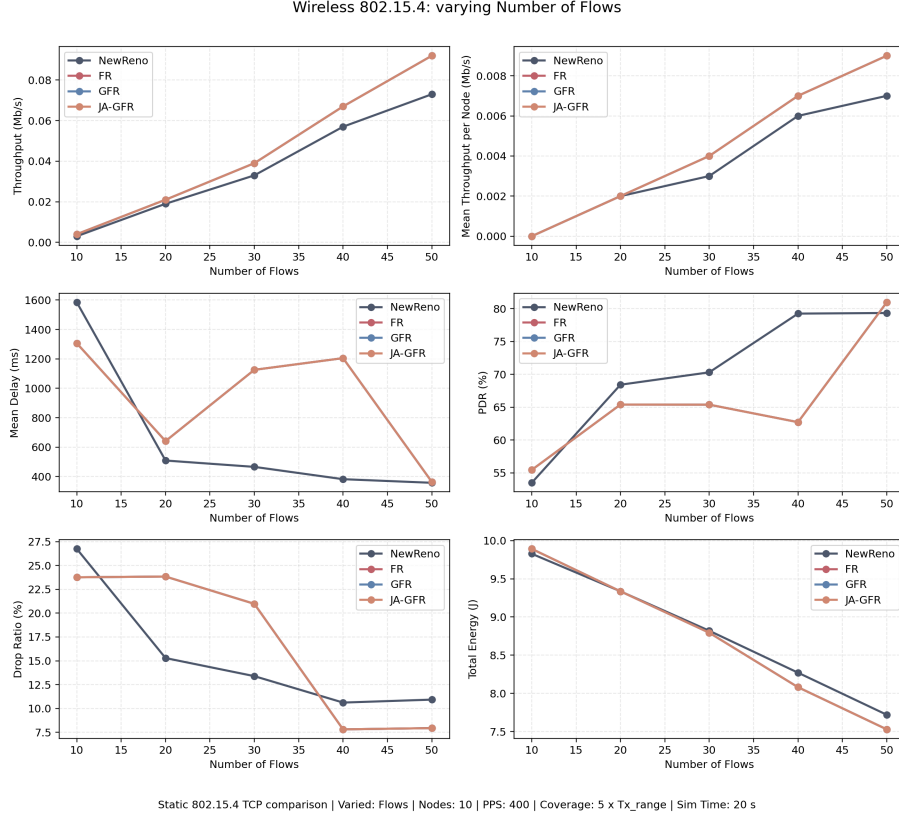


Figure 15: Performance comparison while varying the number of flows.

7.4 Varying Number of Packets

7.4.1 Parameters under Varying

Packets per second was varied as 100, 200, 300, 400, 500 when the number of nodes was 10, the number of flows was 50, the coverage area was $5 \times Tx_range$, and the simulation time was 20 s.

7.4.2 Result

Packets/s	Reno	FR	GFR	JA-GFR
100	0.083	0.086	0.086	0.086
200	0.088	0.084	0.084	0.084
300	0.081	0.086	0.086	0.086
400	0.073	0.092	0.092	0.092
500	0.072	0.078	0.078	0.078

Table 39: Throughput (Mbps) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	318.729	341.517	341.517	341.517
200	308.309	440.961	440.961	440.961
300	322.480	408.348	408.348	408.348
400	357.595	363.640	363.640	363.640
500	361.077	368.430	368.430	368.430

Table 40: Mean Delay (ms) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	82.329	79.222	79.222	79.222
200	81.660	75.496	75.496	75.496
300	80.801	79.042	79.042	79.042
400	79.341	80.970	80.970	80.970
500	78.767	79.366	79.366	79.366

Table 41: Packet Delivery Ratio (%) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	0.008	0.009	0.009	0.009
200	0.009	0.008	0.008	0.008
300	0.008	0.009	0.009	0.009
400	0.007	0.009	0.009	0.009
500	0.007	0.008	0.008	0.008

Table 42: Mean Throughput per Node (Mbps) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	7.740	8.604	8.604	8.604
200	8.172	13.524	13.524	13.524
300	10.152	10.927	10.927	10.927
400	10.928	7.939	7.939	7.939
500	10.578	9.437	9.437	9.437

Table 43: Packet Drop Ratio (%) vs Packets per Second

Packets/s	Reno	FR	GFR	JA-GFR
100	7.673	7.588	7.588	7.588
200	7.608	7.674	7.674	7.674
300	7.693	7.563	7.563	7.563
400	7.720	7.529	7.529	7.529
500	7.744	7.731	7.731	7.731

Table 44: Total Energy Consumption (J) vs Packets per Second

7.4.3 Graph

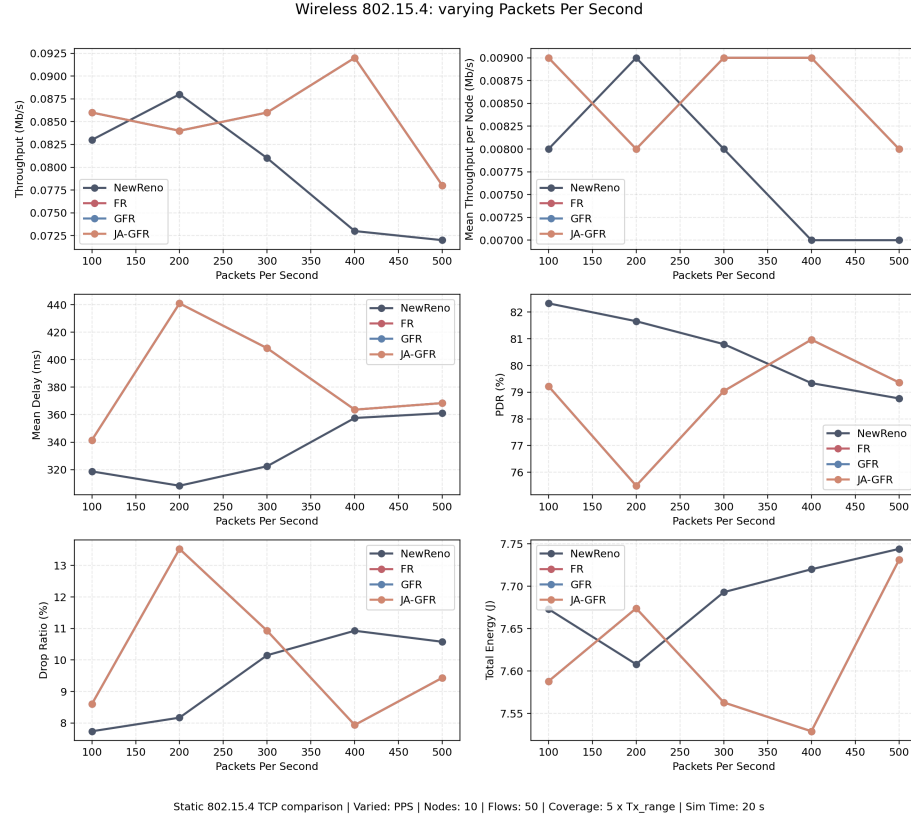


Figure 16: Performance comparison while varying packets per second.

7.5 Varying Coverage Area

7.5.1 Parameters under Varying

Coverage area was varied by changing one side of the square field as Tx_range , $2 \times Tx_range$, $3 \times Tx_range$, $4 \times Tx_range$, $5 \times Tx_range$ when the number of nodes was 10, the number of flows was 50, the packet rate was 400 packets/s, and the simulation time was 20 s.

7.5.2 Result

Coverage	Reno	FR	GFR	JA-GFR
1	0.011	0.011	0.011	0.011
2	0.024	0.035	0.035	0.035
3	0.035	0.052	0.052	0.052
4	0.051	0.071	0.071	0.071
5	0.073	0.092	0.092	0.092

Table 45: Throughput (Mbps) vs Coverage Multiplier

Coverage	Reno	FR	GFR	JA-GFR
1	756.466	756.466	756.466	756.466
2	397.002	582.075	582.075	582.075
3	500.427	595.988	595.988	595.988
4	480.939	454.353	454.353	454.353
5	357.595	363.640	363.640	363.640

Table 46: Mean Delay (ms) vs Coverage Multiplier

Coverage	Reno	FR	GFR	JA-GFR
1	31.754	31.754	31.754	31.754
2	49.796	53.479	53.479	53.479
3	62.861	70.326	70.326	70.326
4	74.040	64.112	64.112	64.112
5	79.341	80.970	80.970	80.970

Table 47: Packet Delivery Ratio (%) vs Coverage Multiplier

Coverage	Reno	FR	GFR	JA-GFR
1	0.001	0.001	0.001	0.001
2	0.002	0.003	0.003	0.003
3	0.004	0.005	0.005	0.005
4	0.005	0.007	0.007	0.007
5	0.007	0.009	0.009	0.009

Table 48: Mean Throughput per Node (Mbps) vs Coverage Multiplier

Coverage	Reno	FR	GFR	JA-GFR
1	26.316	26.316	26.316	26.316
2	22.177	21.703	21.703	21.703
3	19.471	15.721	15.721	15.721
4	12.828	15.140	15.140	15.140
5	10.928	7.939	7.939	7.939

Table 49: Packet Drop Ratio (%) vs Coverage Multiplier

Coverage	Reno	FR	GFR	JA-GFR
1	10.072	10.072	10.072	10.072
2	9.174	9.022	9.022	9.022
3	8.922	8.641	8.641	8.641
4	8.105	7.847	7.847	7.847
5	7.720	7.529	7.529	7.529

Table 50: Total Energy Consumption (J) vs Coverage Multiplier

7.5.3 Graph

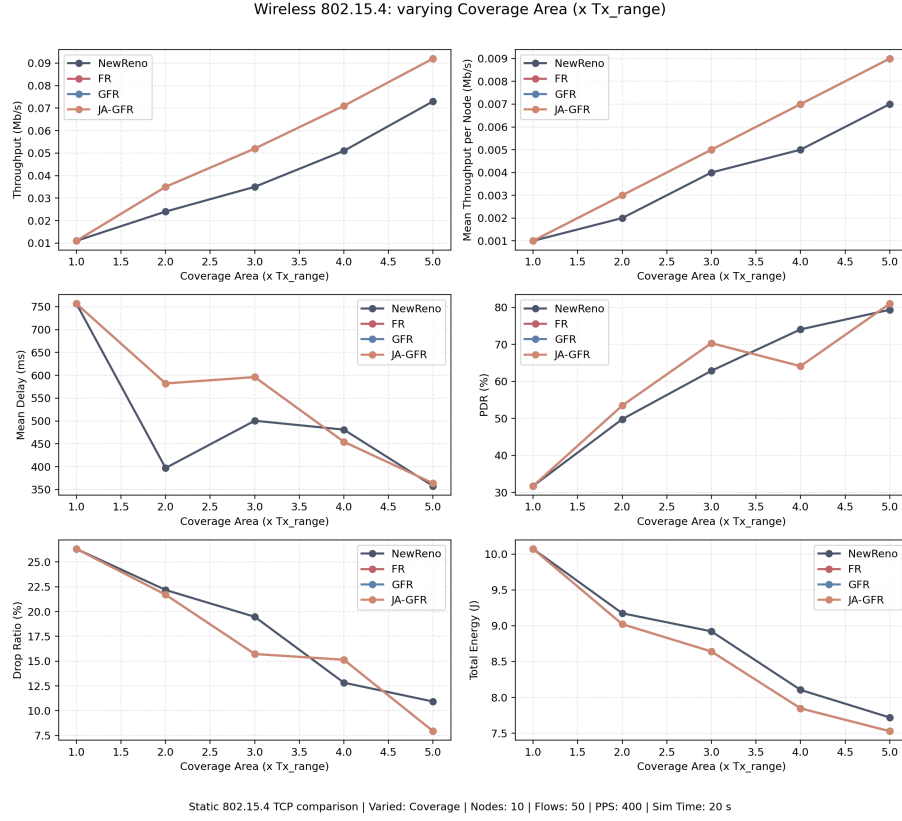


Figure 17: Performance comparison while varying the coverage area.

7.6 Summary Findings

In my setup, the packet size was only 40 bytes and the transmission range of each node was 40 meters. The field size was based on the selected coverage multiplier, so larger coverage values created more **multi-hop communication** between source and destination pairs.

- **The network is highly constrained.** IEEE 802.15.4 has a very low raw data rate, so the available bandwidth is limited from the beginning.
- **The offered load becomes too high very quickly.** With 40-byte packets at 400 packets/s, one flow already generates

$$400 \times 40 \times 8 = 128000\text{bps} = 12.8\text{kbps}.$$

For 50 flows, the one-way offered load becomes

$$50 \times 12.8 = 640\text{kbps},$$

and with ACK traffic included, the total demand becomes even higher than what 802.15.4 can realistically support.

- **Multi-hop forwarding reduces effective throughput.** Because the topology is static and the node range is small, packets often travel over multiple hops before reaching the destination.
- **6LoWPAN forwarding adds extra overhead.** I used 6LoWPAN mesh-under forwarding, so the same packet may consume channel capacity several times along the path.
- **This explains the very low TCP throughput.** Much of the channel time is spent on contention, buffering, ACKs, and forwarding, rather than useful end-to-end TCP delivery.
- **FR, GFR, and JA-GFR do not show the same benefit here.** These algorithms were designed mainly for high bandwidth-delay wired links, where ACK-based bandwidth estimation is more meaningful.
- **In this 802.15.4 network, bandwidth estimation is less useful.** The network has very low bandwidth, strong contention, and heavy forwarding overhead, so more aggressive recovery does not necessarily improve performance.

8 Discussion

Figures shows that my results do not completely match the results reported in the original paper. This is an important point in this reproduction. The original work was implemented in **ns-2**, while my implementation was done in **ns-3**. Even if the same general scenario is used, the two simulators do not behave exactly the same. Differences in TCP implementation, queue handling, timing, and default settings can all affect the final goodput values.

FR, GFR, and JA-GFR are better suited to the wired high-BDP scenario described in the paper. In a mobile wireless802.11 MANET, changing RTT, contention, and route instability and also in a heavily constrained static wireless 802.15.4 multi-hop network make these methods less effective.